eBay-style Proxy Bidding with Hidden Reserve Price 50

EDSL shared-state replication note

October 10, 2026

Abstract

This standalone report documents the sequential proxy bidding condition from the archived llm-auction experiment. It gives the auction's shared-state machine, the local EDSL launch path, and descriptive results from the recorded run. The result is a new model run, not a replay of archived responses.

1 Auction and run design

eBay-style Proxy Bidding with Hidden Reserve Price 50. The condition uses 3 bidders, private values, 2 auction repetitions, model gpt-4, and temperature 0.5. The archived prompts and rule template are taken from `rule\_template/V10/ebay\_t3\_proxy\_with\_hidden\_reserve.txt`.

Item	Status
Auction condition	ebay_reserve_50
Model	gpt-4
Bidders	3
Requested auctions	2
Settled auctions	2
Completed decisions	60
Failed answers	0
State events	123

Table 1: Run coverage and state event counts.

2 Shared-state machine

Each auction repetition has an isolated scope. The machine accepts bidder actions, exposes only its public view during play, and computes settlement in its close transition. This prevents actions in concurrent auctions from mixing. The diagram below was generated by the current EDSL `Machine.show_diagram()` method.

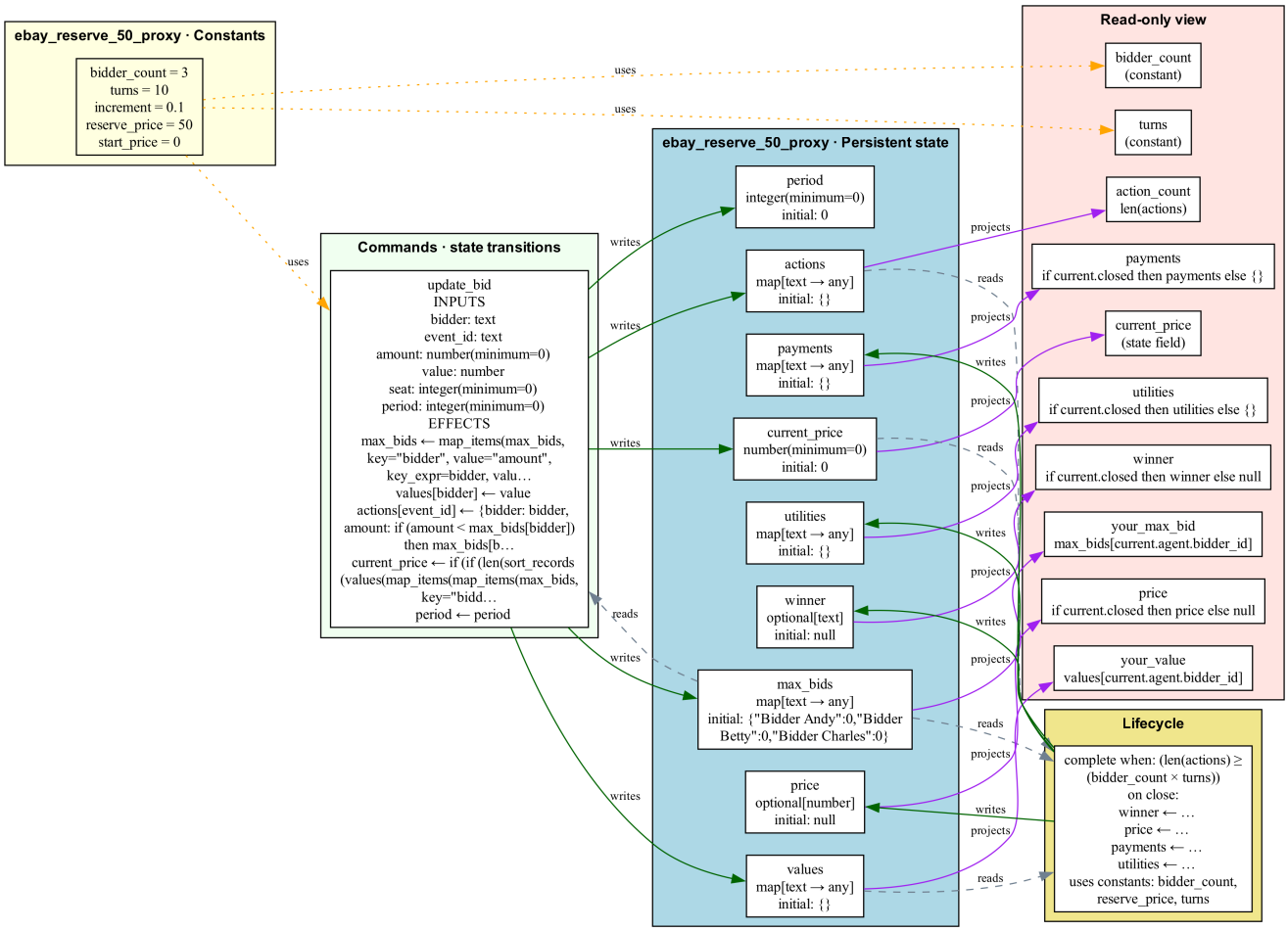


Figure 1: State, command, view, and close lifecycle for this condition.

2.1 Mathematical form

Write the machine as $M = (C, X, A, V, \delta, \kappa)$, with constants C, internal state X, action interface A, public view V, action transition δ, and close transition κ. The scope map assigns independent states x_r to auction repetitions r, so $\delta(x_r, a)$ cannot mutate $x_{r'}$ when $r \neq r'$. For sealed bids, $x = (B, z)$ stores bid records $B = \{(i, s_i, v_i, c, b_i)\}$ by bidder id; the open view reports only the number of submissions. At close, bids are sorted by $(-b_i, s_i)$, the rule selects rank $k \in \{1, 2, 3\}$ (or rank one for all-pay), and payments are $p_i = b_i$ for every bidder in all-pay, otherwise $p_i = q$ for the winner and zero for others. Utility is $u_i = \tilde{v}_i - p_i$, where $\tilde{v}_i$ is the private or common component specified by the condition.

For proxy bidding, the internal state includes maximum bids m_i and public price p. A submitted amount a_i updates $m_i \leftarrow \max(m_i, a_i)$; if the two largest maxima are $m_1 \geq m_2$, then the displayed price is $\min(m_1, m_2 + \Delta)$ when $m_2 > 0$, otherwise the start price. The winner is the top maximum if the final displayed price reaches the reserve. Each serial EDSL interview reads the latest public price before committing its action.

2.2 Definition used for this condition

The implementation below is the actual Python machine factory. The archived configuration supplies the number of bidders, payment rule, value model, reserve, and timing constants. The close transition ranks

bids by amount and stable seat, then records payments and utilities. For proxy auctions, serial live reads expose the updated public price while each bidder's maximum remains private.

```
def build_proxy_machine(config: dict[str, Any]) -> Machine:
    """Build an eBay/Amazon proxy-bid machine with serial public price updates."""
    auction, rule, value_cfg = config["auction"], config["rule"], config["value"]
    count, turns = auction["number_agents"], rule["turns"]
    names = [f"Bidder {n}" for n in NAMES[:count]]
    initial_bids = {name: 0 for name in names}
    # A candidate maximum map includes the current agent's submitted maximum.
    valid_amount = choose(
        arg("amount") < field("max_bids").get(arg("bidder")),
        field("max_bids").get(arg("bidder")), arg("amount"),
    )
    updated_bids = map_items(
        field("max_bids"), key="bidder", value="amount",
        key_expr=local("bidder"),
        value_expr=choose(local("bidder") == arg("bidder"), valid_amount, local("amount")),
    )
    ranked = reduce(
        "sort_records",
        map_items(updated_bids, key="bidder", value="amount",
                  key_expr=local("bidder"),
                  value_expr=record(bidder=local("bidder"), amount=local("amount"))).values(),
        fields=("amount", "bidder"), descending=(True, False),
    )
    top = ranked.first()
    second = choose(ranked.length() > 1, ranked.at(1).get("amount"), 0)
    current_price = choose(
        second > 0,
        choose(top.get("amount") < second + constant("increment"),
               top.get("amount"), second + constant("increment")),
        constant("start_price"),
    )
    sale = current_price >= constant("reserve_price")
    winner = choose(sale, top.get("bidder"), None)
    final_ranked = reduce(
        "sort_records",
        map_items(field("max_bids"), key="bidder", value="amount",
                  key_expr=local("bidder"),
                  value_expr=record(bidder=local("bidder"), amount=local("amount"))).values(),
        fields=("amount", "bidder"), descending=(True, False),
    )
    final_sale = field("current_price") >= constant("reserve_price")
    final_winner = choose(final_sale, final_ranked.first().get("bidder"), None)
    payments = map_items(
        field("max_bids"), key="bidder", value="amount",
        key_expr=local("bidder"),
        value_expr=choose(local("bidder") == final_winner, field("current_price"), 0),
    )
    utilities = map_items(
        field("max_bids"), key="bidder", value="amount",
        key_expr=local("bidder"),
        value_expr=choose(
            local("bidder") == final_winner,
            field("values").get(local("bidder")) - field("current_price"), 0,
        ),
```

```
    )
    # amounts may only be raised; EDSL serial order makes each update visible
    # to the next bidder in a round.
    return Machine(
        name=f"{config['experiment']['name']}_proxy",
        constants={"bidder_count": count, "turns": turns,
                   "increment": value_cfg["increment"],
                   "reserve_price": rule["reserve_price"],
                   "start_price": rule.get("start_price", 0),
                   "soft_close": bool(rule.get("closing", False))},
        fields={
            "max_bids": state_field(StateType.map(), initial_bids),
            "values": state_field(StateType.map(), {}),
            "actions": state_field(StateType.map(), {}),
            "period": state_field(StateType.integer(minimum=0), 0),
            "current_price": state_field(StateType.number(minimum=0), rule.get("start_price", 0)
                ),
            "winner": state_field(StateType.optional(StateType.text()), None),
            "price": state_field(StateType.optional(StateType.number()), None),
            "payments": state_field(StateType.map(), {}),
            "utilities": state_field(StateType.map(), {}),
        },
        commands={"update_bid": Command(
            inputs={"bidder": StateType.text(), "event_id": StateType.text(),
                    "amount": StateType.number(minimum=0),
                    "value": StateType.number(), "seat": StateType.integer(minimum=0),
                    "period": StateType.integer(minimum=0)},
            effects=(
                assign("max_bids", updated_bids),
                put("values", arg("bidder"), arg("value")),
                put("actions", arg("event_id"), record(
                    bidder=arg("bidder"), amount=valid_amount, period=arg("period"),
                    seat=arg("seat"))),
                assign("current_price", current_price),
                assign("period", arg("period")),
            ),
        )},
        view={
            "bidder_count": constant("bidder_count"),
            "turns": constant("turns"),
            "current_price": field("current_price"),
            "your_max_bid": field("max_bids").get(current("agent.bidder_id", default=""), 0),
            "your_value": field("values").get(current("agent.bidder_id", default=""), 0),
            "action_count": field("actions").length(),
            "winner": choose(current("closed"), field("winner"), None),
            "price": choose(current("closed"), field("price"), None),
            "payments": choose(current("closed"), field("payments"), {}),
            "utilities": choose(current("closed"), field("utilities"), {}),
        },
        complete_when=field("actions").length() >= constant("bidder_count") * constant("turns")
            ,
        close_effects=(assign("winner", final_winner),
                       assign("price", choose(final_sale, field("current_price"), None)),
                       assign("payments", payments), assign("utilities", utilities)),
    )
```

3 Local EDSL execution

The EDSL runner executes on the local machine and calls the provider named in the YAML using the configured API key. No localhost model server is part of this run. The launch command for this condition is:

```
python -m edsl_replication.shared_state_auctions \\
  configs/experiments/02_03_ebay_reserve_50.yaml \\
  --output edsl_replication/studies/edsl\_replication/studies/ebay\_reserve\_50/pilot\_02\
      _auctions/raw_results.json
```

The raw result contains the EDSL interviews, state map, machine event history, and exit snapshots. CSV exports retain bidder-turn and auction-level observations.

4 Observed results

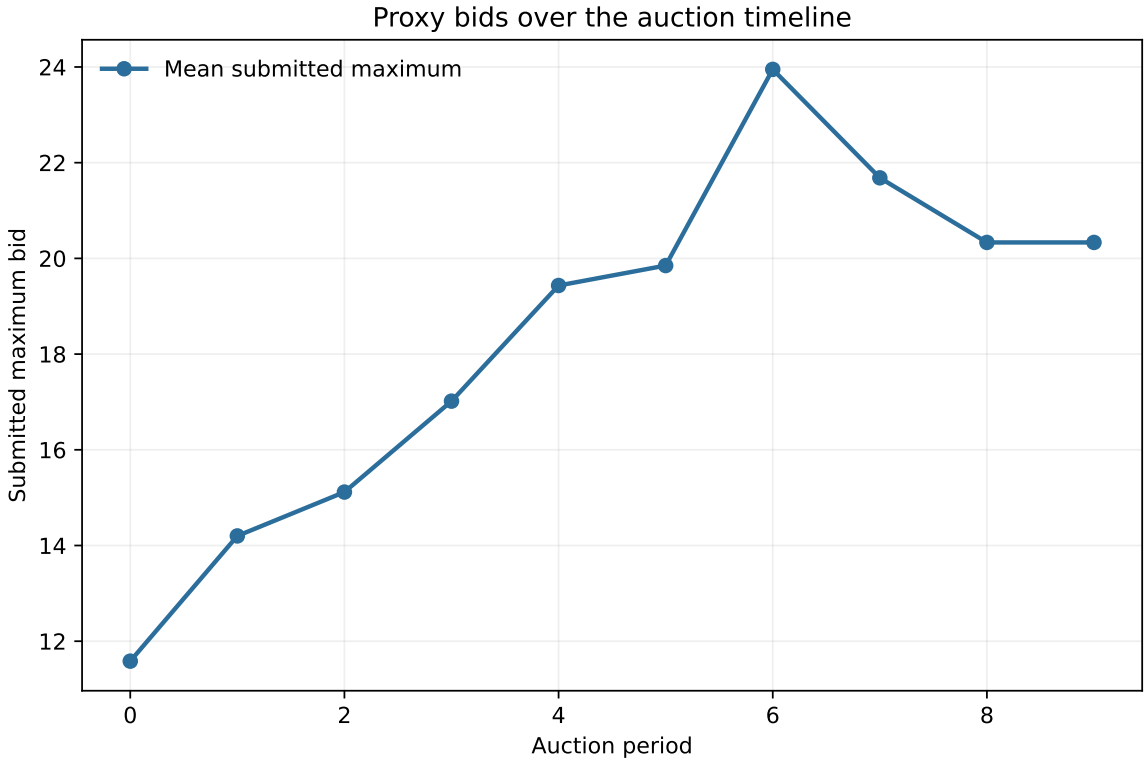


Figure 2: Descriptive result visualization for this condition.

Mean bidder value: 26.667; Mean bid or final maximum: 28.050; Mean bid-value gap: 1.383; Sale rate: 0.000.

The plotted pattern is descriptive. These runs use the archived prompt and condition settings but a newly executed EDSL shared-state protocol; differences in question structure and execution order can affect model responses.

5 Protocol limits

- Randomized within-period order is seeded; it is fixed before inference.

- Minimum bid increment is described in prompts; numeric bids are not grid-validated.